



LINGUAGEM DE PROGRAMAÇÃO ORIENTADA A OBJETOS C++

Composição e Agregação

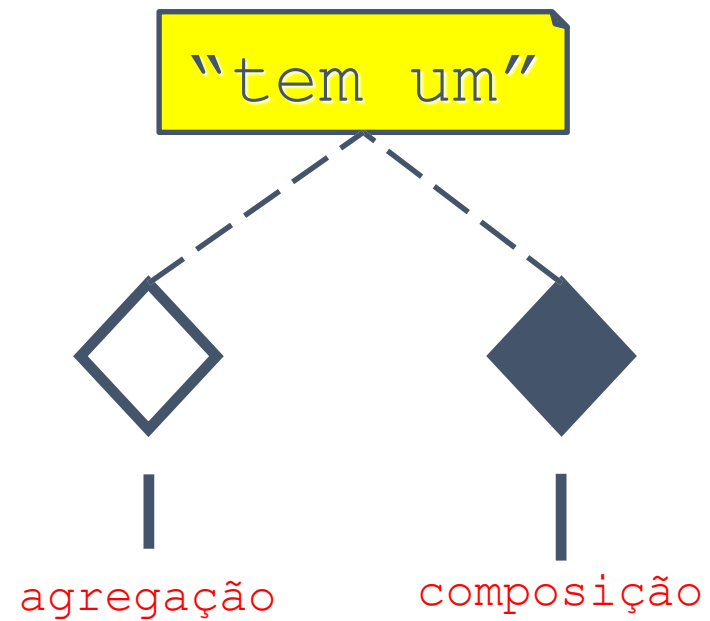
Conteúdo da Aula

- Associação
 - Agregação e Composição

Associação

- Ela descreve um vínculo que ocorre entre classes ou até mesmo que uma classe esteja vinculada a si própria ou que uma associação seja compartilhada por mais de uma classe. Representamos as associações por meio de retas que ligam as classes envolvidas, essas setas podem ou não possuir setas nas extremidades indicando a navegabilidade da associação, ou seja, o sentido em que as informações são passadas entre as classes.
- Quando falamos de *agregação* e *composição* estamos falando de casos especiais de tipos de associação entre classes.

Associação



Agregação

- É um tipo especial de associação onde tenta-se demonstrar que as informações de um objeto (chamado objeto-todo) precisam ser complementados pelas informações contidas em um ou mais objetos de outra classe (chamados objetos-parte); conhecemos como todo/parte.
- Se eu tiver um sistema de cadastro de times, preciso cadastras várias pessoas para agregar os times, assim, cada pessoa pode agregar um time, nenhum time, ou vários times. A pessoa é independente do time, mas agrega valor a ele.

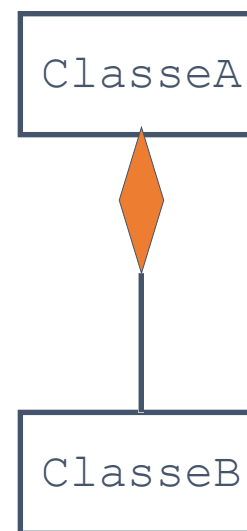
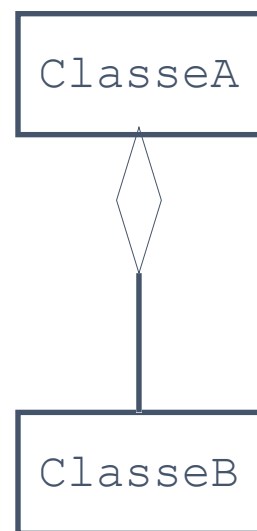
Composição

- Pode-se dizer que composição é uma variação da agregação. Uma composição tenta representar também uma relação todo - parte. No entanto, na composição o objeto-pai (todo) é responsável por criar e destruir suas partes. Em uma composição um mesmo objeto-parte não pode se associar a mais de um objeto-pai.
- Toda vez que dizemos que a relação entre duas classe é de **composição** estamos dizendo que uma dessas classe (a Parte) está contida na outra (o Todo) e a parte não vive/não existe sem o todo.
- Sendo assim, toda vez que destruímos o todo, a parte que é única e exclusiva do todo se vai junto.
- Por esse motivo: a parte *está contida* no todo. Quando se joga o todo fora, a parte estava dentro e se vai junto.

Composição

- É necessário que exista pelo menos um item em uma nota fiscal para que a nota fiscal exista. E um item só irá existir se a nota fiscal existir.
- Não há diferença na implementação e sim no comportamento.

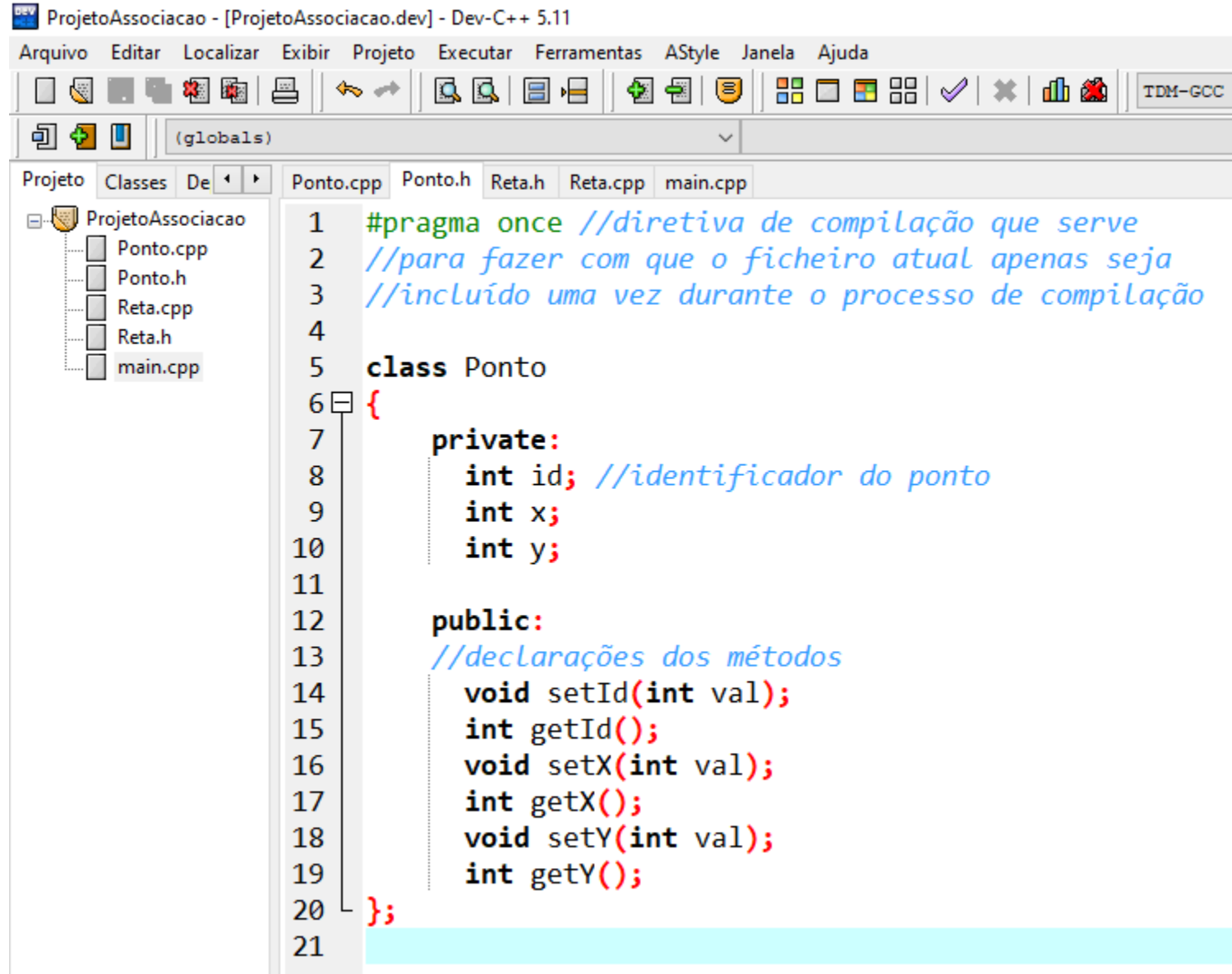
Agregação e Composição



Exemplo

Ponto		Reta
- int id - int x - int y		- int id - Ponto *a - Ponto *b
+ <u>setId</u> + <u>setX</u> + <u>setY</u> + <u>getX</u> + <u>getY</u>		+ <u>setId</u> + <u>setA</u> + <u>setB</u> + <u>getA</u> + <u>getB</u>

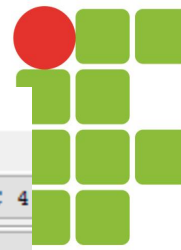
Ponto.h



```
ProjetoAssociacao - [ProjetoAssociacao.dev] - Dev-C++ 5.11
Arquivo  Editar  Localizar  Exibir  Projeto  Executar  Ferramentas  AStyle  Janela  Ajuda
(globals)
Projeto  Classes  De  Ponto.cpp  Ponto.h  Reta.h  Reta.cpp  main.cpp

ProjetoAssociacao
├── Ponto.cpp
├── Ponto.h
├── Reta.cpp
├── Reta.h
└── main.cpp

1  #pragma once //diretiva de compilação que serve
2  //para fazer com que o ficheiro atual apenas seja
3  //incluído uma vez durante o processo de compilação
4
5  class Ponto
6  {
7      private:
8          int id; //identificador do ponto
9          int x;
10         int y;
11
12     public:
13         //declarações dos métodos
14         void setId(int val);
15         int getId();
16         void setX(int val);
17         int getX();
18         void setY(int val);
19         int getY();
20 };
21
```



Ponto.cpp

[*] Ponto.cpp

```
1  #pragma once
2  #include "Ponto.h"
3
4  void Ponto::setId(int val)
5  {
6      id = val;
7  }
8
9  int Ponto::getId()
10 {
11     return id;
12 }
13
14 void Ponto::setX(int val)
15 {
16     x = val;
17 }
18
```

```
18
19 int Ponto:: getX()
20 {
21     return x;
22 }
23
24 void Ponto:: setY(int val)
25 {
26     y = val;
27 }
28
29 int Ponto:: getY()
30 {
31     return y;
32 }
33
```

Reta.h

```
1  #pragma once
2  #include "Ponto.h"
3
4  class Reta
5  {
6      private:
7          int id; //identificador da reta
8          Ponto *a; //Associação entre a classe Reta e Ponto
9          Ponto *b; //A Reta possui um Ponto a e um Ponto b.
10
11      public:
12          void setId(int val);
13          int getId();
14          void setA(Ponto *val);
15          Ponto *getA();
16          void setB(Ponto *val);
17          Ponto *getB();
18          void print(); //declaração do método (assinatura)
19  };
20
```

Reta.cpp

[*] Ponto.cpp Ponto.h Reta.cpp [*] Reta.h

```
1  #pragma once
2
3  #include <iostream>
4  #include "Reta.h"
5
6  void Reta::setId(int val)
7  {
8      id = val;
9  }
10
11 int Reta::getId()
12 {
13     return id;
14 }
15
16 void Reta::setA(Ponto *val)
17 {
18     a = val;
19 }
20
```

```
20
21 Ponto *Reta::getA()
22 {
23     return a;
24 }
25
26 void Reta::setB(Ponto *val)
27 {
28     b = val;
29 }
30
31 Ponto *Reta::getB()
32 {
33     return b;
34 }
35
36 void Reta::print()
37 {
38     std::cout
39         << "Reta " << id
40         << ": (" << a->getX() << ", " << a->getY() << ") - ("
41         << b->getX() << ", " << b->getY() << ")" << std::endl;
42 }
```

main.cpp

[*] Ponto.cpp Ponto.h [*] Reta.cpp [*] Reta.h main.cpp

```
1 #include <iostream>
2 #include "Ponto.h"
3 #include "Reta.h"
4
5 int main()
6 {
7     Ponto *p1 = new Ponto();
8     p1->setId(1); //chamada de função/método
9     p1->setX(50);
10    p1->setY(100);
11
12    Ponto *p2 = new Ponto();
13    p2->setId(2);
14    p2->setX(200);
15    p2->setY(200);
16
17    std::cout << "Ponto 1: (x,y) = "
18        << p1->getX() << ", " << p1->getY() << std::endl;
19
20    std::cout << "Ponto 2: (x,y) = "
21        << p2->getX() << ", " << p2->getY() << std::endl;
22
```

main.cpp

```
22
23   Reta *r1 = new Reta();
24   r1->setId(3);
25   r1->setA(p1);
26   r1->setB(p2);
27
28   std::cout
29       << "Reta " << r1->getId() << ": (x0,y0) - (x1,y1) = ("
30       << r1->getA()->getX() << ", " << r1->getA()->getY()
31       << ") - ("
32       << r1->getB()->getX() << ", " << r1->getB()->getY()
33       << ")." << std::endl << std::endl;
34
35   r1->print(); //chama o método print do objeto r1
36
37   return 0;
38 }
```

main.cpp

 C:\Users\SÚrgio\Dropbox\2020_2\01 - Prog2\Aulas\Aula 6 - Projeto-Associatião\ProjetoAssociatião

```
Ponto 1: (x,y) = 50, 100
```

```
Ponto 2: (x,y) = 200, 200
```

```
Reta 3: (x0,y0) - (x1,y1) = (50, 100) - (200, 200).
```

```
Reta 3: (50, 100) - (200, 200)
```

```
-----
```

```
Process exited after 0.3163 seconds with return value 0
```

```
Pressione qualquer tecla para continuar. . . _
```